

Zustand Interview Questions & Answers with Code Examples

■ 1. What is Zustand?

Answer:

Zustand is a lightweight state-management library for React. It allows us to create a centralized store without the boilerplate commonly associated with Redux.

It uses a simple API based on a store containing state and actions.

```
npm install zustand
```

Basic example:

```
import { create } from "zustand";

type CounterStore = {
  count: number;
  increment: () => void;
  decrement: () => void;
};

const useCounterStore = create<CounterStore>((set) => ({
  count: 0,

  increment: () =>
    set((state) => ({
      count: state.count + 1,
    })),

  decrement: () =>
    set((state) => ({
      count: state.count - 1,
    })),
}));
```

Component:

```
function Counter() {
  const count = useCounterStore((state) => state.count);
  const increment = useCounterStore((state) => state.increment);

  return (
    <div>
      <p>{count}</p>
      <button onClick={increment}>+</button>
    </div>
  );
}
```

■ 2. Why would you use Zustand instead of Redux?

Answer:

I would use Zustand when I need centralized state management but want a simpler API and less boilerplate.

Zustand

```

    const useStore = create((set) => ({
count: 0,

increment: () =>
    set((state) => ({
    count: state.count + 1,
})),
}));

```

Redux Toolkit

Redux Toolkit generally involves:

Store configuration

Slices

Reducers

Actions

Provider

Dispatch/selectors

Zustand can be much more lightweight.

Interview answer:

"I prefer Zustand for applications where global state is required but Redux's structure and boilerplate aren't necessary. For large applications with complex state transitions, strict patterns, middleware requirements, or extensive Redux ecosystem usage, Redux Toolkit can still be a better choice."

■ 3. How do you create a Zustand store?

```

Use the create() function.
import { create } from "zustand";

const useUserStore = create((set) => ({
user: null,

setUser: (user) =>
    set({
user,
}),

logout: () =>
    set({
user: null,
}),
}));

```

Usage:

```

function Profile() {
const user = useUserStore((state) => state.user);
const logout = useUserStore((state) => state.logout);

return (
<>
<h2>{user?.name}</h2>

```

```

    <button onClick={logout}>Logout</button>
  </>
);
}

```

■ 4. What is set in Zustand?

set is used to update the store.

```

const useStore = create((set) => ({
  count: 0,

  increment: () => {
    set({
      count: 1,
    });
  },
}));

```

More commonly, we derive the new state from the previous state:

```

increment: () =>
  set((state) => ({
    count: state.count + 1,
  })))

```

This is important when the new value depends on the previous value.

■ 5. What is get in Zustand?

get allows us to access the current state from inside the store.

```

const useCartStore = create((set, get) => ({
  items: [],

  addItem: (item) => {
    const items = get().items;

    set({
      items: [...items, item],
    });
  },
}));

```

Here:

`get().items`

returns the current items.

■ 6. What is the difference between set and get?

Example:

```

const useStore = create((set, get) => ({
  count: 0,

  increment: () => {
    const currentCount = get().count;

    set({

```

```
count: currentCount + 1,  
  });  
},  
));
```

■ 7. How do you update nested state in Zustand?

Suppose:

```
const useUserStore = create((set) => ({  
  user: {  
    name: "Priyanka",  
    address: {  
      city: "Bangalore",  
    },  
  },  
  updateCity: (city) =>  
    set((state) => ({  
      user: {  
        ...state.user,  
        address: {  
          ...state.user.address,  
          city,  
        },  
      },  
    })),  
}));
```

We need to preserve the existing object properties using the spread operator.

■ 8. Does Zustand use immutability?

Yes.

You should generally update objects and arrays immutably.

```
set((state) => ({  
  todos: [  
    ...state.todos,  
    newTodo,  
  ],  
}));
```

Instead of:

```
state.todos.push(newTodo);
```

■ 9. What is a selector in Zustand?

A selector allows a component to subscribe only to the state it needs.

```
const count = useStore((state) => state.count);
```

Instead of:

```
const store = useStore();
```

Using selectors is generally better because it can reduce unnecessary component re-renders.

Example:

```
function Counter() {
  const count = useStore((state) => state.count);

  return <p>{count}</p>;
}
```

The component subscribes specifically to count.

■ 10. Why are selectors important in Zustand?

Consider:

```
const { count, user, cart, settings } = useStore();
```

The component subscribes to the entire selected store object.

A better approach is:

```
const count = useStore((state) => state.count);
// If the component only needs count, it doesn't need to subscribe to unrelated state.
```

Senior-level answer:

"I use fine-grained selectors in Zustand to minimize subscriptions and avoid unnecessary re-renders."

■ 11. How do you select multiple values from Zustand?

You can use useShallow.

```
import { useShallow } from "zustand/react/shallow";

const { count, user } = useStore(
  useShallow((state) => ({
    count: state.count,
    user: state.user,
  })))
);
```

This is useful when selecting multiple values as an object.

■ 12. How do you handle arrays in Zustand?

Example:

```
const useTodoStore = create((set) => ({
  todos: [],

  addTodo: (todo) =>
    set((state) => ({
      todos: [...state.todos, todo],
    })),

  removeTodo: (id) =>
    set((state) => ({
      todos: state.todos.filter(
        (todo) => todo.id !== id
      ),
    })),
```

```
toggleTodo: (id) =>
  set((state) => ({
    todos: state.todos.map((todo) =>
      todo.id === id
        ? { ...todo, completed: !todo.completed }
        : todo
    )
  })),
  ));
```

■ 13. How do you use Zustand with TypeScript?

Define an interface/type for the store.

```
import { create } from "zustand";

type User = {
  id: string;
  name: string;
  email: string;
};

type UserStore = {
  user: User | null;
  setUser: (user: User) => void;
  logout: () => void;
};

const useUserStore = create<UserStore>((set) => ({
  user: null,

  setUser: (user) =>
    set({
      user,
    }),

  logout: () =>
    set({
      user: null,
    }),
}));
```

This provides type safety for state and actions.

■ 14. Can Zustand handle asynchronous operations?

Yes.

You can perform API calls inside store actions.

```
const useUserStore = create((set) => ({
```

```

users: [],
loading: false,
error: null,

    fetchUsers: async () => {
    set({
loading: true,
error: null,

    });
try {
    const response = await fetch("/api/users");
    const data = await response.json();
    set({
users: data,
loading: false,

    });
} catch (error) {
    set({
error: "Failed to fetch users",
loading: false,

    });
}
},
    ));

```

Component:

```

function Users() {
const users = useUserStore((state) => state.users);
const loading = useUserStore((state) => state.loading);
const fetchUsers = useUserStore((state) => state.fetchUsers);

useEffect(() => {
fetchUsers();

}, [fetchUsers]);

if (loading) return <p>Loading...</p>;

return (
<>
    {users.map((user) => (
    <p key={user.id}>{user.name}</p>
    ))}
</>
);
}

```

■ 15. Where should API calls be placed in Zustand?

There are two common approaches.

Approach 1 — Store action

```

const useStore = create((set) => ({
  users: [],

  fetchUsers: async () => {
    const response = await fetch("/api/users");
    const users = await response.json();

    set({ users });
  },

}));

```

Approach 2 — Separate service layer

For larger applications, I prefer:

components/

stores/

services/

api/

Example:

```

// services/userService.ts

export async function fetchUsers() {
  const response = await fetch("/api/users");

  if (!response.ok) {
    throw new Error("Failed to fetch users");
  }

  return response.json();
}

```

Then:

```

const useUserStore = create((set) => ({
  users: [],

  fetchUsers: async () => {
    const users = await fetchUsers();

    set({ users });
  },

}));

```

For a large production application, separating API/service logic can improve maintainability.

■ 16. Does Zustand replace React Query/TanStack Query?

No.

This is a very important interview question.

Zustand is primarily for client/application state.
TanStack Query is designed for server state.

Client state

Examples:

theme

sidebar open/closed

modal state

selected filters

shopping cart

UI preferences

Server state

Examples:

users from API

products

orders

posts

comments

Interview answer:

"I don't consider Zustand and TanStack Query direct replacements. I use Zustand for client-side global state and TanStack Query for server-state fetching, caching, synchronization, and mutations."

■ 17. Can Zustand persist state after page refresh?

Yes.

Use the persist middleware.

```
import { create } from "zustand";
import { persist } from "zustand/middleware";

const useAuthStore = create(
  persist(
    (set) => ({
      user: null,

      login: (user) =>
        set({
          user,
        }),

      logout: () =>
        set({
          user: null,
        }),
    })
  )
);
```

The state is persisted using browser storage by default.

■ 18. What is Zustand persist middleware?

persist automatically stores selected Zustand state in persistent storage.

Example:

```

    persist(
      (set) => ({
theme: "dark",

setTheme: (theme) =>
  set({ theme }),
}),
{
name: "theme-storage",
}
)

```

After refreshing the page, the persisted state can be restored.

■ 19. How do you persist only part of the store?

Use `partialize`.

```

const useStore = create(
  persist(
    (set) => ({
user: null,
token: null,
loading: false,

setUser: (user) => set({ user }),
setToken: (token) => set({ token }),
}),
{
name: "auth-storage",

partialize: (state) => ({
  user: state.user,
}),
}
)
);

```

Only user is persisted.

■ 20. How do you reset a Zustand store?

You can create a reset action.

```

const initialState = {
count: 0,
name: "",
};

const useStore = create((set) => ({
...initialState,

```

```

setCount: (count) =>
  set({ count }),
setName: (name) =>
  set({ name }),
reset: () =>
  set(initialState),
});

```

Usage:

```

const reset = useStore((state) => state.reset);

<button onClick={reset}>Reset</button>

```

■ 21. How do you create multiple Zustand stores?

It's usually better to split unrelated state.
stores/

■■■ authStore.ts

■■■ cartStore.ts

■■■ uiStore.ts

■■■ notificationStore.ts

Example:

```

// authStore.ts

export const useAuthStore = create((set) => ({
  user: null,

  login: (user) => set({ user }),

  logout: () => set({ user: null }),
}));

// cartStore.ts

export const useCartStore = create((set) => ({
  items: [],

  addItem: (item) =>
    set((state) => ({
      items: [...state.items, item],
    })),
}));

```

This keeps responsibilities separated.

■ 22. How do you access Zustand state outside React components?

One of Zustand's useful features is that the store provides APIs such as `getState()`.

```
const user = useAuthStore.getState().user;
```

You can also call an action:

```
useAuthStore.getState().logout();
```

This can be useful in places where React hooks cannot be used.

For example:

```

const user = useAuthStore.getState().user;

if (!user) {
  // handle unauthenticated state
}

```

■ 23. Can you use Zustand in event handlers?

Yes.

```

function LogoutButton() {
  const logout = useAuthStore((state) => state.logout);

  const handleLogout = () => {
    logout();
  };

  return (
    <button onClick={handleLogout}>
      Logout
    </button>
  );
}

```

■ 24. What is middleware in Zustand?

Middleware extends store functionality.

Common middleware includes:

persist

devtools

immer

subscribeWithSelector

Example:

```

import { devtools } from "zustand/middleware";

const useStore = create(
  devtools((set) => ({
    count: 0,

    increment: () =>
      set((state) => ({
        count: state.count + 1,
      })),
  })),
);

```

■ 25. How do you use Zustand with Redux DevTools?

Use the devtools middleware.

```

import { create } from "zustand";
import { devtools } from "zustand/middleware";

const useStore = create(

```

```

    devtools((set) => ({
count: 0,

increment: () =>
    set(
      (state) => ({
        count: state.count + 1,
      }),
    false,
    "counter/increment"
  ),
  )))
);

```

This makes debugging state changes easier.

■ 26. What is Immer and how can it be used with Zustand?

Immer allows us to write mutation-like code while maintaining immutable updates internally.

```

import { create } from "zustand";
import { immer } from "zustand/middleware/immer";

const useStore = create(
  immer((set) => ({
user: {
name: "Priyanka",
address: {
city: "Bangalore",
  },
  },
updateCity: (city) =>
  set((state) => {
    state.user.address.city = city;
  }),
  )))
);

```

Without Immer:

```

set((state) => ({
user: {
  ...state.user,
address: {
  ...state.user.address,
city,
  },
  },
  }));

```

Immer makes deeply nested updates easier to read.

■ 27. How do you handle authentication using Zustand?

Example:

```
type AuthStore = {
  user: User | null;
  isAuthenticated: boolean;

  login: (user: User) => void;
  logout: () => void;
};

const useAuthStore = create<AuthStore>((set) => ({
  user: null,
  isAuthenticated: false,

  login: (user) =>
    set({
      user,
      isAuthenticated: true,
    }),
  logout: () =>
    set({
      user: null,
      isAuthenticated: false,
    }),
}));
```

Component:

```
function Navbar() {
  const user = useAuthStore((state) => state.user);
  const logout = useAuthStore((state) => state.logout);

  return (
    <nav>
      {user ? (
        <>
          <span>{user.name}</span>
          <button onClick={logout}>Logout</button>
        </>
      ) : (
        <button>Login</button>
      )}
    </nav>
  );
}
```

Important security point: Don't casually persist sensitive authentication secrets in localStorage; token handling should follow your application's security architecture, often using secure, HttpOnly cookies.

■ 28. How do you implement a shopping cart using Zustand?

This is a very common practical interview question.

```
type Product = {
  id: number;
  name: string;
  price: number;
};

type CartStore = {
  items: Product[];
  addItem: (product: Product) => void;
  removeItem: (id: number) => void;
  clearCart: () => void;
};

const useCartStore = create<CartStore>((set) => ({
  items: [],

  addItem: (product) =>
    set((state) => ({
      items: [...state.items, product],
    })),

  removeItem: (id) =>
    set((state) => ({
      items: state.items.filter(
        (item) => item.id !== id
      ),
    })),

  clearCart: () =>
    set({
      items: [],
    }),
}));

Component:

function Cart() {
  const items = useCartStore((state) => state.items);
  const removeItem = useCartStore(
    (state) => state.removeItem
  );

  return (
    <div>
      {items.map((item) => (
        <div key={item.id}>
          {item.name} - ${item.price}

          <button
```

```

onClick={() => removeItem(item.id)}
>
Remove
</button>
</div>
)}}
</div>
);
}

```

■ 29. How would you implement cart quantity?

A better production implementation stores quantity rather than duplicate products.

```

type CartItem = {
product: Product;
quantity: number;
};

type CartStore = {
items: CartItem[];

addItem: (product: Product) => void;
increaseQuantity: (id: number) => void;
decreaseQuantity: (id: number) => void;
};

Add item:
addItem: (product) =>
  set((state) => {
    const existing = state.items.find(
      (item) => item.product.id === product.id
    );
    if (existing) {
      return {
        items: state.items.map((item) =>
          item.product.id === product.id
            ? {
                ...item,
                quantity: item.quantity + 1,
              }
            : item
        ),
      };
    }
  })
}

```

```

    return {
items: [
    ...state.items,
{
product,
quantity: 1,
    },
],
};
}),

```

This is a good example to discuss in a senior interview because it demonstrates state modeling rather than just Zustand syntax.

■ 30. How do you calculate derived state in Zustand?

For example, total cart price.

You don't necessarily need to store total separately.

```

const total = useCartStore((state) =>
state.items.reduce(
(sum, item) =>
sum + item.product.price * item.quantity,
0
)
);

```

This avoids having two sources of truth.

Senior-level principle:

"I avoid storing values that can be reliably derived from existing state."

■ 31. How do you subscribe to Zustand state changes?

Zustand provides `subscribe()`.

```

const unsubscribe = useStore.subscribe(
state) => {
console.log("State changed:", state);
}
);

```

You can later unsubscribe:

```
unsubscribe();
```

This is useful for side effects and integrations outside normal component rendering.

■ 32. What is `subscribeWithSelector`?

It allows subscriptions to specific pieces of state.

```
import { subscribeWithSelector } from "zustand/middleware";
```

```

const useStore = create(
subscribeWithSelector((set) => ({

```

```
count: 0,
```

```
)))
```

```
);
```

Then:

```
const unsubscribe = useStore.subscribe(  
  (state) => state.count,  
  (count) => {
```

```
  console.log("Count changed:", count);
```

```
  }
```

```
);
```

This avoids reacting to unrelated store changes.

■ 33. Can Zustand be used with Next.js?

Yes, but you need to understand the Server Component / Client Component boundary.

For example:

"use client";

```
import { useCounterStore } from "@stores/counterStore";
```

```
export default function Counter() {  
  const count = useCounterStore(  
    (state) => state.count
```

```
);
```

```
  return <p>{count}</p>;
```

```
}
```

A component using the Zustand React hook should be a Client Component.

■ 34. What is an important Zustand consideration with Next.js Server Components?

Do not treat a shared module-level Zustand store as request-specific server state.

In Next.js applications, server rendering introduces concerns around:

Request isolation

Hydration

Server/client boundaries

Persisted browser state

For client state, a common pattern is to keep the Zustand store on the client side and use appropriate hydration/persistence strategies.

Senior interview answer:

"With Next.js App Router, I pay attention to the server/client boundary. I avoid putting request-specific state into a global

■ 35. Zustand vs Context API?

Example Context:

```
const ThemeContext = createContext(null);
```

Zustand:

```
const useThemeStore = create((set) => ({
```

```

theme: "light",

toggleTheme: () =>
  set((state) => ({
theme:
  state.theme === "light"
? "dark"
: "light",
})),
});

```

■ 36. Zustand vs Redux Toolkit?

A strong interview answer:

"Redux Toolkit provides a more opinionated architecture with reducers, actions, middleware and a predictable state flow. Zustand is more minimal and gives me direct access to state and actions through a store. For a smaller or moderately complex application, Zustand can reduce boilerplate. For large teams or applications that benefit from strict conventions, extensive middleware, complex workflows, or an existing Redux ecosystem, Redux Toolkit can be preferable."

■ 37. Does Zustand use reducers?

Not necessarily.

You can directly define actions:

```

const useStore = create((set) => ({
count: 0,

increment: () =>
  set((state) => ({
    count: state.count + 1,
  })),
}));

```

You can also design reducer-style logic if your application benefits from it, but Zustand does not require reducers.

■ 38. How would you structure Zustand in a large application?

I would avoid putting everything into one giant store.

Example:

```

src/
■ ■ ■ stores/
■ ■ ■ ■ auth/
■ ■ ■ ■ ■ authStore.ts
■ ■ ■ ■ cart/
■ ■ ■ ■ ■ cartStore.ts
■ ■ ■ ■ ui/
■ ■ ■ ■ ■ uiStore.ts

```

- ■■■ notification/
- ■■■ notificationStore.ts
-
- services/
- ■■■ authService.ts
- ■■■ userService.ts
- ■■■ productService.ts
-
- components/
- app/

This provides better separation of concerns.

■ 39. What are common mistakes when using Zustand?

Mistake 1 — Putting everything into one store

Bad:

```
const useStore = create((set) => ({
  user: {},
  products: [],
  cart: [],
  notifications: [],
  theme: "dark",
  modal: false,
  ...
}));
```

Better:

```
authStore
cartStore
productStore
uiStore
```

Mistake 2 — Selecting the entire store unnecessarily

Avoid:

```
const store = useStore();
```

Prefer:

```
const user = useStore((state) => state.user);
```

Mistake 3 — Duplicating derived state

Avoid:

```
{
  items: [],
  total: 100
}
```

if total can be calculated from items.

Mistake 4 — Using Zustand for everything

Don't automatically put API/server state into Zustand.

Consider:

Zustand → client/application state

TanStack Query → server state

React local state → component-only state

■ 40. How would you optimize a Zustand application?

I would focus on:

1. Use selectors

```
const count = useStore(
  (state) => state.count
);
```

2. Avoid huge stores

Split stores by domain.

3. Use shallow comparison where appropriate

```
useStore(
  useShallow((state) => ({
    count: state.count,
    user: state.user,
  })))
);
```

4. Keep derived state derived

Don't unnecessarily duplicate state.

5. Separate server state

Use TanStack Query where appropriate.

6. Avoid unnecessary persistence

Only persist the state that actually needs persistence.

■ 41. Real-world Senior React interview question

"You have a React application with Redux. The application has authentication, shopping cart, UI state and API data. Would you migrate everything to Zustand?"

Strong answer:

"No. I wouldn't migrate everything just because Zustand has less boilerplate. First I'd categorize the state. Authentication/

This is much stronger than saying:

"Zustand is better than Redux."

■ 42. Practical interview question: Build a theme store

```
import { create } from "zustand";
```

```

type Theme = "light" | "dark";

type ThemeStore = {
  theme: Theme;
  toggleTheme: () => void;
};

const useThemeStore = create<ThemeStore>((set) => ({
  theme: "light",

  toggleTheme: () =>
    set((state) => ({
      theme:
        state.theme === "light"
        ? "dark"
        : "light",
    })),
}));

export default useThemeStore;

```

Component:

```

function ThemeButton() {
  const theme = useThemeStore(
    (state) => state.theme
  );

  const toggleTheme = useThemeStore(
    (state) => state.toggleTheme
  );

  return (
    <button onClick={toggleTheme}>
      Current theme: {theme}
    </button>
  );
}

```

■ 43. Practical interview question: Build a modal store

```

type ModalStore = {
  isOpen: boolean;
  openModal: () => void;
  closeModal: () => void;
};

const useModalStore = create<ModalStore>((set) => ({
  isOpen: false,

  openModal: () =>
    set({

```

```

isOpen: true,
  }},

  closeModal: () =>
    set({
      isOpen: false,
    }),
  }));

```

Component:

```

function App() {
  const isOpen = useModalStore(
    (state) => state.isOpen
  );

  const openModal = useModalStore(
    (state) => state.openModal
  );

  const closeModal = useModalStore(
    (state) => state.closeModal
  );

  return (
    <>
      <button onClick={openModal}>
        Open
      </button>

      {isOpen && (
        <div>
          <h2>Modal</h2>

          <button onClick={closeModal}>
            Close
          </button>
        </div>
      )}
    </>
  );
}

```

■ 44. How would you explain Zustand in 30 seconds?

Use this in an interview:

"Zustand is a lightweight state-management library for React. It lets us create a centralized store containing state and actions."